

本报告基于 5703 CS60-1 项目当前 baseline (v1_5000_100ep_stage1_dinov3, PSNR 20.76) 与 2017-2025 年 FiveK 文献调研,提出一套针对当前 frozen DINOv3 + RWKV bottleneck pipeline 的 decoder 完整重构方案。目标:把 PSNR 从 20.76 dB 提升到 25.0+ dB。

一、当前现状与基线

1.1 任务与数据

任务为图像色彩重映射 (image color enhancement) ,数据集 MIT-Adobe FiveK,目标 Expert C 的人工修图结果。输入输出皆为 RGB sRGB,空间分辨率 480×480。当前实验使用 5000 张样本,80/10 /10 切分,训练 100 epoch。

1.2 流水线结构

模块	实现	状态	输出形状
Encoder	DINOv3 ViT-B (patch=16, emb=768)	冻结	(B, 900, 768)
Bottleneck	RWKVBottleneckV1 (hidden=384, 6 层)	可训	(B, 900, 384)
Decoder	CNN PixelShuffle 渐进上采样	可训	(B, 3, 480, 480)
Pipeline 收尾	clamp(img + delta, 0, 1)	—	—

1.3 当前 decoder 简述

入口 LayerNorm + Linear 把 384 维 token 投影到 base_channels=128,reshape 成 (B, 128, 30, 30) 特征图。然后 4 次 PixelShuffle 上采样,通道按 [128, 96, 64, 48, 24] 衰减。每个上采样段融合一个 1×1 卷积投影后的原图作为 skip。最终 head 通过 ChanLayerNorm + 3×3 + GELU + 1×1 输出 3 通道,经 tanh × 0.5 得到限制在 ±0.5 的残差 delta。

1.4 当前测试集成绩

指标	值
test/psnr	20.76
test/ssim	0.7528
test/lpips	0.1220
test/nima	5.59
test/clip	4.53
试验代号	v1_5000_100ep_stage1_dinov3

二、当前 decoder 的根本性问题

本节分析的是范式与结构层面的限制,而非超参可调修的表层问题。

2.1 范式错误:逐像素 RGB 残差回归

当前 decoder 把"图像色彩重映射"当作任意像素到任意像素的回归任务。然而 FiveK 修图差异在像素分布上的本质是 photometric 的、低维的、空间低频的:几乎所有 input $\rightarrow$ Expert C 的差异可以用一组色调曲线 + 局部仿射近似。逐像素独立回归相当于把一个本质 12 参数能描述的变换,丢给 10M 参数的网络去重新学习,样本效率极差,且与任务先验对抗。

2.2 输出能力被 $\tanh \times 0.5$ 钉死

代码 cnn_decoder.py 第 193 行: `return torch.tanh(self.head(x)) * self.residual_scale`。`residual_scale` 默认 0.5,意味着任何需要超过 0.5 像素值位移的区域 (如阴影抬升、高光裁剪重建) 在数学上都到不了。即便 pipeline 后续会做 clamp, $\tanh$ 之内的饱和已经把上限钉死。

2.3 入口信息瓶颈

384 $\rightarrow$ 128 一次性压缩 (cnn_decoder.py:111),通道按 [128, 96, 64, 48, 24] 衰减到全分辨率仅余 24 通道。低层信息被压扁,末端表达力不足以承载 RGB 输出。

2.4 缺乏全局机制

RWKV 在 token 域提供了 $O(N)$ 全局上下文,但 decoder 一旦回到 2D 全是 3×3 卷积,全局信息被丢弃。FiveK 任务里 "这张是日落场景所以推暖色"、"这张过曝所以压亮度" 这类语义判断必须在 decoder 内部仍可用,纯卷积做不到。

2.5 图像旁路过弱

当前 img_proj 仅是 1×1 卷积投影 raw image (img_proj_channels=3),没有学习任何低层视觉特征。decoder 实质拿到的是"语义 token + 缩放后的原图像素",没有学到的图像低层结构。

2.6 5K 样本下的过拟合风险

10M 可训练参数全部投入逐像素回归,5K 样本数量级下不可避免地过拟合到训练集像素噪声,而非学到真正的修图先验。对比之下,CSRNet 用 37K 参数在 FiveK 上达到 25.21 dB, SepLUT 用 119K 达到 25.47 dB——证明此任务参数效率比容量更重要。

三、文献调研:FiveK SOTA 全景

3.1 PaperWithCode 校对的 leaderboard

下表为 MIT-Adobe FiveK Expert C, 480p, sRGB $\rightarrow$ sRGB 标准评估,4500/500 train/test 切分 (Bychkovsky split)。

方法	年份	PSNR ↑	SSIM	参数量	范式
HDRNet	2017	23.71-24.32	0.91	480K	双边网格仿射
DPE	2018	23.75	0.908	3.4M	UNet 残差 (像素回归)
DeepUPE	2019	21.88-23.48	0.85-0.91	1.0M	光照分解
DeepLPF	2020	24.43	0.937	800K	椭圆/曲线 graduated 滤波
3D-LUT	2020	25.07-25.29	0.93	600K	K=3 基底 33 LUT
CSRNet	2020	25.21	0.923	37K	全局向量调制的 per-pixel MLP
StarEnhancer	2021	25.73	0.937	小	多风格条件 head
Adalnt-LUT	2022	25.49	0.926	619K	自适应采样 3D-LUT
SepLUT	2022	25.47	0.921	119K	1D + 3D LUT 串联
4D-LUT	2023	24.96	0.924	924K	上下文感知 LUT
RSFNet	2023	25.34	0.938	—	区域专属白盒滤波
ICELUT	2024	25.27	0.918	<1MB 存储	全 LUT 推理
NamedCurves	2024	25.59	0.936	小	命名颜色曲线 + 注意力
LUTwithBGrid	2024	25.66	0.930	463K	3D-LUT + 双边网格
SVDLUT	2025	25.76	0.931	160K	SVD 分解空间 LUT
DiffRetouch	2024	26.21	0.944	大	多步 diffusion
当前 baseline	—	20.76	0.753	10M	像素残差回归

3.2 关键洞察

第一,参数化变换全面碾压像素回归。前 10 名里 9 个属于 LUT、双边网格、曲线、per-pixel 仿射这类"输出变换参数,而非输出像素"的范式。

第二,参数效率比容量更重要。CSRNet 37K、SepLUT 119K、SVDLUT 160K——三个 25+ dB 方法的参数量都比当前 baseline 小两个数量级。原因:正确的归纳偏置降低了搜索空间,小模型反而更容易学到先验。

第三,LUT + 双边网格混合是 2024 年验证过的方向。LUTwithBGrid (ECCV 2024) 已经把这条路走通,25.66 dB。本方案与之同构,但使用 ViT 语义特征替换原文的小 CNN backbone。

第四,diffusion 不推荐。DiffRetouch 26.21 dB 但参数大、推理慢,与项目"frozen encoder 高效推理"目标不符。

3.3 Decoder 设计分类

范式	代表方法	FiveK PSNR	5K 样本友好度
像素残差/UNet	DPE / NAFNet decoder	22-24	差
局部参数变换	DeepLPF / DeepUPE	23.5-25	好
全局 LUT	3D-LUT / Adalnt / SepLUT	25.0-25.7	极好
双边网格	HDRNet	24-25.7	好
LUT + 双边混合	LUTwithBGrid / 本方案	25.4-25.8	好
Diffusion	DiffRetouch	26.2	差 (需大数据)

四、推荐架构:TokenLUT-BG

4.1 设计原则

第一,主路径走参数化变换,不再做逐像素 RGB 回归。

第二,双层结构:全局 3D-LUT 处理整体色调,空间双边网格处理局部修饰。

第三,保留一条小残差通道作为兜底 (高光裁剪重建、降噪等 LUT 与仿射处理不了的情况)。

第四,额外加一条 raw image 光度旁路 (GuideNN) 弥补 DINOv3 训练时引入的色不变性。

第五,与现有 pipeline 对外接口兼容,只在内部走另一条路径。

4.2 总体架构

decoder 接受 (tokens, h, w, img) 三元组,内部分三个 head 并联,输出完整图像而非残差。

```
tokens (B, 900, 384) ----[reshape]----> tokens_grid (B, 384, 30, 30)
                                \--[mean pool]---> global_vec (B, 384)
img (B, 3, 480, 480)
```

Head A : Global 3D-LUT

Head B : Bilateral Grid Affine

Head C : GuideNN (raw image 光度旁路)

```
I_lut = trilinear_lookup(weighted_LUT, img)
coeffs = bilateral_slice(grid, guide_map)
I_aff = M(x,y) · [I_lut_R, I_lut_G, I_lut_B, 1]^T
out    = clamp(I_lut + gate · (I_aff - I_lut), 0, 1)
```

4.3 Head A:全局 3D-LUT

- 基底 LUT:nn.Parameter,形状 (M=3, 3, 17, 17, 17),可训。其中 1 个初始化为 identity (即 $LUT[i,j,k] = (i,j,k)/16$),另外 2 个初始化为 small random。
- 权重 MLP:Linear(384 → 64) → GELU → Linear(64 → 3) → Softmax,作用于 mean-pooled global_vec。
- 光度增强 token (额外):在 global_vec 上 concat raw image 的 mean RGB、std RGB、9-bin RGB 直方图共 24 维,弥补 DINOv3 色不变性。
- 应用:fused_LUT = $\sum w \boxtimes LUT_m$,然后用 F.grid_sample 5D mode 做三线性查表 (无需 CUDA 自定义算子)。
- 基底 K=3 的依据:Zeng 2020 原文消融,K=3 在 FiveK 上是甜点位,K>3 在小数据集上反而退化。
- 17 的依据:AdaInt / SepLUT 同样使用 17。33 在 5K 样本下顶点严重欠覆盖。
- 预期单分支贡献:+3 ~ +4 dB。

4.4 Head B:双边网格仿射 (Bilateral Affine Grid)

- 网格生成:在 tokens_grid (384, 30, 30) 上做一个 3×3 卷积输出 (B, 12, 8, 30, 30),然后 reshape 成 (B, 12, 8, 30, 30)。这是一个 30×30 空间分辨率、8 个 luma bin、每格 12 维 (3×4 仿射) 的 bilateral grid。

- 初始化:卷积权重 zero-init,使初始网格全零 → 对 LUT 输出无影响。
- Slicing:通过 GuideNN 产生的 luma guide 在 grid 上做三线性采样,产生 per-pixel 的 (3, 4) 仿射矩阵 $M(x,y)$ 。
- 应用: $I_{\text{aff}} = \text{stack}(a \cdot R + b \cdot G + c \cdot B + d \text{ for each color})$,作用在 I_{lut} 上 (注意是在 LUT 后的图像上,而非原图)。
- 网格分辨率选择: 30×30 与 ViT token 分辨率天然对齐,避免上采样。luma bin = 8 是 HDRNet 默认。
- 预期单分支贡献:+0.8 ~ +1.5 dB (在 LUT 之上)。

4.5 Head C:GuideNN (raw image 光度旁路)

- 结构: $\text{Conv}1 \times 1(3 \rightarrow 8) \rightarrow \text{ReLU} \rightarrow \text{Conv}1 \times 1(8 \rightarrow 1) \rightarrow \text{Sigmoid}$ 。整个 head 大约 50 个参数。
- 作用对象:raw input image,产出 (B, 1, 480, 480) 的 luma guide,用于 Head B 的 slicing。
- 存在意义:DINOv3 在自监督训练时大量使用 color jitter,模型被刻意训练成"对色温/曝光不敏感"。这与 FiveK 任务正相反——FiveK 就是要预测色温/曝光的修改。GuideNN 让网络有一条直通 raw image 的光度通路。
- 初始化:正常初始化,无特殊处理 (因为它仅作 guide,不直接出图)。

4.6 三支组合

- gate:可学习标量,初始化为 0,经 sigmoid 后位于 [0, 1]。最终输出 = $\text{clamp}(I_{\text{lut}} + \text{gate}(I_{\text{aff}} - I_{\text{lut}}), 0, 1)$ 。
- 训练动力学:gate 初始化为 0 时,输出完全等于 I_{lut} ;Head A (LUT) 先学到位之后 gate 才会自然增大,Head B 才开始接管局部修饰。
- 可选扩展:若实测过拟合,可在 gate 上加 L2 正则。

4.7 关键超参与初始化

项	值	依据
LUT 基底数 K	3	Zeng 2020 消融
LUT 分辨率	17	文献共识,5K 下不超 17
双边网格空间分辨率	30×30	与 ViT token 对齐
双边网格 luma bin	8	HDRNet 默认
GuideNN 通道	$3 \rightarrow 8 \rightarrow 1$	极轻量
LUT 初始化	1 identity + 2 small random	训练第一步即 identity,稳
网格初始化	zero	初始无空间修饰,LUT 主导
gate 初始化	0 (经 sigmoid 后 ~0.5,实际通过偏置控制为 ~0)	训练初期纯 LUT
总参数估计	~250-300K	对比当前 10M,小两个数量级

4.8 输出契约变更

当前 pipeline (pipeline.py:76) 是 $(x + \text{decoder_output}).\text{clamp}(0, 1)$, 即 decoder 输出残差。新 decoder 自然输出完整图像。两种方案:

方案一 (推荐): 让新 decoder 暴露一个属性 `predict_residual = False`, pipeline 检测到后改走 `decoder_output.clamp(0, 1)`。现有 CNNDecoder 保持 `predict_residual = True`, 行为不变。

方案二: 新 decoder 内部计算 `residual = full_image - input_image` 返回。这样接口完全兼容, 但浪费一次减法。

采用方案一。pipeline.py 改动约 5 行, 完全向后兼容。

五、实现路线

每个 PR 都新建一个 decoder 实现 + yaml, 通过 `decoder=...` 切换; 不修改现有 `cnn_decoder.py`, 实验间互不干扰。

PR1 仅 Head A (3D-LUT only)

- 新增 `src/models/decoders/lut_decoder.py`
- 新增 `configs/decoder/lut.yaml`
- 修改 `src/models/pipeline.py` 支持 `predict_residual=False`
- 工程量约 250 行 (含 utility)
- 预期 PSNR: 24.0 ~ 24.5
- 验证目标: 确认 LUT 范式在 frozen DINOv3 上能否 work。如果不到 23 dB, 问题在上游 (RWKV / DINOv3 信号), 不是 decoder。

PR2 + Head C (GuideNN) + 光度增强 token

- 在 `lut_decoder.py` 加 GuideNN
- 在 LUT weight MLP 输入端 concat 24 维光度 token
- 工程量约 50 行
- 预期 PSNR: 24.3 ~ 24.8
- 此 PR 单独存在的意义: 验证 DINOv3 色不变性问题是否成立。如果 PR2 比 PR1 高 0.5 dB+, 证实假设。

PR3 + Head B (Bilateral Affine Grid)

- 新增 `src/models/decoders/tokenlut_bg.py` (基于 PR2 扩展)
- 新增 `configs/decoder/tokenlut_bg.yaml`
- 实现 bilateral slicing (用 `F.grid_sample 5D`)
- 工程量约 150 行
- 预期 PSNR: 25.0 ~ 25.3

PR4 联合调参 + 消融 + 最终复现

- 调 LR、weight decay、loss weight、warmup
- 跑全部消融实验: 仅 A、A+B、A+C、A+B+C、不同 LUT 数 ($K=1, 3, 5$)、不同网格分辨率
- 工程量主要在训练/记录, 不在代码
- 预期 PSNR: 25.3 ~ 25.5

预期增益总览

阶段	PSNR	与 baseline 差	信心
当前 baseline	20.76	—	—
PR1 (LUT only)	24.0-24.5	+3.2-3.7	高
PR2 (+ GuideNN)	24.3-24.8	+3.5-4.0	中-高
PR3 (+ BGrid)	25.0-25.3	+4.2-4.5	中-高
PR4 (调参)	25.3-25.5	+4.5-4.7	中

六、风险与缓解

6.1 DINOv3 色不变性可能压低 LUT 信号质量

DINO 系列自监督训练大量用 color jitter,模型被刻意训练成对色温曝光不敏感。这正是 FiveK 任务想预测的修改。

缓解:Head C (GuideNN) + 24 维光度增强 token 共同补这块。

诊断:若 PR2 没显著超越 PR1,说明 ViT 范式本身不适合此任务,需要做对照实验:换一个从头训的小 C NN encoder 测 LUT 效果。

6.2 RWKV bottleneck 可能丢失光度信息

RWKV bottleneck 训练目标 (combined loss) 不显式保留 raw photometric statistics。

缓解:同 6.1。光度旁路与光度增强 token 都不依赖 bottleneck 输出。

诊断:训完 PR1 后做特征 probe,看 bottleneck 输出对 RGB 均值的预测精度,即可判断信息是否丢失。

6.3 5K 样本可能不够

所有引用的 SOTA PSNR 数字基于 4500 train / 500 test 标准切分。需要确认本项目的 5000 是仅 train 还是 train+val+test。

缓解 1:若实际 train 数 < 4500,所有预期 PSNR 数字下调 0.5 ~ 1 dB。

缓解 2:LUT 类方法本就极度参数高效,小数据集下相对优势更大,不会塌。

6.4 工程风险

- bilateral slicing 是否需要 CUDA 算子:不需要。F.grid_sample 5D 模式直接做三线性采样,纯 PyTorch 可实现。LUTwithBGrid、AdaInt 重新实现时也都用了这条路径。
- 训练稳定性:三支并联储在初期可能出现 gate 早期被推高、LUT 还没学好就被覆盖。通过零初始化网格 + gate sigmoid 偏置初始为大负数 (如 -3) 可避免。
- 梯度回流:三线性查表通过 grid_sample 的天然可微性回传,无特殊处理。

6.5 评估协议差异

不同 FiveK 论文使用 MATLAB imresize / PIL bicubic / OpenCV INTER_AREA 等差异会带来 0.3 dB 左右抖动。

缓解:与现有 baseline (v1_5000_100ep_stage1_dinov3) 用同一套评估代码对照,所有 PR 之间数字内部可比。

七、验证计划

7.1 主对照表

每个 PR 与 baseline 在同一切分、同一评估代码下比较。

实验	decoder	训练样本	epoch	关键 metric
baseline	cnn (current)	5000	100	PSNR/SSIM/LPIPS
PR1	lut	5000	100	同
PR2	lut + guidenn	5000	100	同
PR3	tokenlut_bg	5000	100	同
PR4 调参	tokenlut_bg	5000	100/150	同

7.2 消融实验 (PR4 阶段)

消融项	说明
K = 1, 3, 5, 8	验证基底数 K=3 是否最优
LUT 13 vs 17 vs 33	验证 17 是否最优
GuideNN on/off	验证 6.1 假设
光度增强 token on/off	验证 6.1 假设
Bilateral grid 16×16 vs 30×30 vs 60×60	验证 30×30 选择
Luma bin 4/8/16	验证 8 是否最优
gate 学习 vs gate=1 固定	验证学习 gate 必要性

7.3 评估指标

主指标 PSNR、SSIM、LPIPS;辅助指标 NIMA、CLIP score (与现 baseline 一致)。如有时间增加 ΔE_ab (CIELab 色差,FiveK 论文常用)。

7.4 训练超参 (起始值)

延续当前 base.yaml 配置 (lr=1e-4, wd=1e-4, AdamW, cosine, warmup 5 epoch),仅在 PR4 阶段调整。Loss 沿用 stage1 (L1 + 0.1 SSIM + 0.01 perceptual)。

八、预期收益与时间线

8.1 PSNR 目标

- 保守目标:24.5 dB (PR3 完成,PR4 不调参)
- 主要目标:25.0 dB (PR4 中等调参)

- 激进目标:25.5 dB (PR4 充分调参,接近 LUTwithBGrid 同档)

8.2 时间估计

PR	实现	训练 (单跑)	累计
PR1	1-2 天	半天 (服务器)	2-3 天
PR2	0.5 天	半天	3-4 天
PR3	1-2 天	半天	5-6 天
PR4	调参 1 周	多次	12-15 天

8.3 失败时的回退

若 PR1 不到 23 dB:暂停 PR2/3,先做诊断实验 (换小 CNN encoder 测 LUT decoder),判断问题源。

若 PR3 不到 24.5 dB:暂停 PR4,做特征 probe 与额外消融,定位 token 信号缺失环节。

九、与现有方案的兼容性

- encoder:不动。
- bottleneck:不动。
- loss:不动 (PR4 可调权重)。
- 数据加载:不动。
- 训练脚本:不动 (通过 hydra 切 decoder)。
- 评估:不动。
- pipeline.py:小改动,支持 predict_residual=False 分支,完全向后兼容。

附录 参考文献与代码

核心方法

- HDRNet (Gharbi 2017 SIGGRAPH): <https://github.com/google/hdrnet>, PyTorch port <https://github.com/creativ/hdrnet-pytorch>
- 3D-LUT (Zeng 2020 TPAMI): <https://github.com/HuiZeng/Image-Adaptive-3DLUT>
- CSRNet (He 2020 ECCV): <https://github.com/hejingwenhejingwen/CSRNet>
- Adalnt-LUT (Yang 2022 CVPR): <https://github.com/ImCharlesY/Adalnt>
- SepLUT (Yang 2022 ECCV): <https://github.com/ImCharlesY/SepLUT>
- LUTwithBGrid (Kim 2024 ECCV): <https://github.com/WontaeaeKim/LUTwithBGrid>
- SVDLUT (2025 arXiv 2508.16121)
- ICELUT (2024 arXiv 2403.19238)
- DiffRetouch (2024 arXiv 2407.03757)
- NamedCurves (Serra 2024 ECCV): <https://github.com/davidsera9/namedcurves>
- DPT (Ranftl 2021 ICCV): <https://arxiv.org/abs/2103.13413>

Benchmark 与协议

- PaperWithCode FiveK 480p: <https://paperswithcode.com/sota/photo-retouching-on-mit-adobe-5k-480p>

项目内文件

- 当前 decoder: src/models/decoders/cnn_decoder.py
- pipeline: src/models/pipeline.py
- DINOv3 encoder: src/models/encoders/dinov3.py
- RWKV bottleneck: src/models/bottlenecks/rwkv_bottleneck.py
- baseline 配置: configs/config.yaml + configs/base.yaml
- baseline 结果: docs/v1-stage 1/v1_5000_100ep_stage1_dinov3_results.json